

HuggingFace를 활용한

FastAPI 인공지능 웹개발 프로젝트

제 1장

인공지능과 Hugging Face의 첫걸음

이 장의 학습 목표

생성형 AI(텍스트, 이미지, 음성 생성)의 핵심 아이디어와 구조를 개략적으로 이해합니다.

Hugging Face가 제공하는 모델, 데이터셋, Spaces에 대한 개념을 학습합니다.

FastAPI를 활용한 AI 웹서버 개발 흐름을 익히는 데 기반을 다집니다.

목 차

제1장 인공지능과 Hugging Face의 첫걸음

★ 이 장의 개요

이 장에서는 인공지능과 생성형 AI의 전반적인 흐름을 이해하고, Hugging Face 플랫폼의 개념과 구조를 소개합니다. 또한 본격적인 프로젝트 개발에 앞서 Python 개발 환경을 설정하고, FastAPI를 활용한 첫 번째 웹 애플리케이션(Hello World)을 만들어보는 실습을 진행합니다.

현대 소프트웨어 개발에서 인공지능(AI)은 더 이상 선택 사항이 아닙니다. 챗봇, 이미지 생성, 음성 인식, 번역 서비스 등 다양한 분야에서 AI 기술이 핵심 역할을 맡고 있으며, 이를 웹 서비스와 결합하는 능력이 현대 개발자에게 필수 역량으로 자리 잡았습니다.

이 교재는 Python 기반의 최신 웹 프레임워크인 FastAPI와 세계 최대 AI 모델 허브인 Hugging Face를 결합하여, 실용적이고 확장 가능한 AI 웹 서비스를 개발하는 방법을 단계별로 안내합니다. 1장에서는 그 기초를 탄탄히 다집니다.

1.1 인공지능, 생성형 AI란 무엇인가?

인공지능(Artificial Intelligence, AI)이라는 개념은 1950년대에 처음 등장했지만, 오늘날 우리가 실생활에서 경험하는 AI는 그 역사가 훨씬 짧고 혁명적인 발전 과정을 거쳤습니다. 특히 2020년대에 들어 '생성형 AI(Generative AI)'라는 새로운 패러다임이 등장하면서 AI의 가능성은 전혀 다른 차원으로 확장되었습니다.

1.1.1 전통적인 AI와 생성형 AI의 차이

전통적인 AI 시스템은 주로 '판별(Discriminative)' 작업에 집중했습니다. 예를 들어, 스팸 메일 분류기는 이메일이 스팸인지 아닌지를 판단하고, 이미지 분류 모델은 사진 속 객체가 고양이인지 개인지를 판별합니다. 이러한 모델들은 주어진 입력에 대해 미리 정의된 범주 중 하나를 선택하는 방식으로 동작합니다.

반면, 생성형 AI는 완전히 새로운 콘텐츠를 '생성(Generate)'합니다. 텍스트, 이미지, 오디오, 비디오 등 기존에 존재하지 않던 새로운 데이터를 만들어내는 것이 핵심 특징입니다. 이를 위해 생성형 AI는 대규모 데이터로부터 데이터의 내재적 분포(Distribution)를 학습하고, 이를 기반으로 새로운 샘플을 생성합니다.

구분	전통적 AI (판별 모델)	생성형 AI (생성 모델)
목적	입력을 분류/예측	새로운 콘텐츠 생성
출력	클래스 레이블, 숫자	텍스트, 이미지, 오디오
학습 방식	지도학습 중심	비지도/자지도 학습
대표 모델	ResNet, BERT(분류)	GPT-4, DALL·E, Stable Diffusion
활용 사례	스팸 필터, 얼굴 인식	ChatGPT, 이미지 생성, 번역
데이터 요구량	상대적으로 적음	수십억 개 이상

생성형 AI의 핵심 구성 요소는 크게 세 가지로 나뉩니다.

- 트랜스포머(Transformer) 아키텍처: 2017년 구글이 발표한 논문 'Attention Is All You Need'에서 제안된 신경망 구조로, 현대 생성형 AI의 근간을 이룹니다. 셀프-어텐션(Self-Attention) 메커니즘을 통해 문장 내 단어들 사이의 관계를 효과적으로 학습합니다.
- 대규모 사전 학습(Large-scale Pre-training): 수십억 개의 파라미터를 갖는 모델을 방대한 데이터로 사전 학습시킵니다. GPT-4는 약 1조 개의 파라미터를 가진 것으로 추정됩니다.
- 파인튜닝 및 프롬프트 엔지니어링(Fine-tuning & Prompt Engineering): 사전 학습된 모델을 특정 도메인이나 작업에 맞게 조정하거나, 적절한 프롬프트를 통해 원하는 출력을 유도합니다.

1.1.2 생성형 AI의 대표 분야: 텍스트, 이미지, 오디오, 멀티모달

생성형 AI는 다루는 데이터 유형에 따라 크게 네 가지 분야로 구분됩니다. 각 분야는 서로 다른 모델 구조와 학습 방법을 채용하지만, 최근에는 여러 모달리티를 동시에 처리하는 멀티모달 AI의 등장으로 경계가 점점 허물어지고 있습니다.

(1) 텍스트 생성 (Text Generation)

텍스트 생성 모델은 가장 먼저 대중화된 생성형 AI 분야입니다. 자연어 처리(NLP) 기술을 기반으로 하며, 입력된 텍스트(프롬프트)를 이해하고 문맥에 맞는 응답을 생성합니다.

대형 언어 모델(LLM, Large Language Model)은 이 분야의 핵심 기술로, 수천억 개의 토큰(단어 혹은 단어 조각)으로 학습됩니다. 대표적으로 OpenAI의 GPT 시리즈, Meta의 LLaMA, Google의 Gemini, Anthropic의 Claude 등이 있습니다.

텍스트 생성 AI의 주요 활용 분야는 다음과 같습니다.

- 대화형 AI (Chatbot, Virtual Assistant)
- 코드 자동 생성 및 디버깅 (GitHub Copilot, Cursor)
- 문서 요약 및 번역
- 창작 콘텐츠 생성 (소설, 시나리오, 마케팅 카피)
- 검색 증강 생성 (RAG, Retrieval-Augmented Generation)

(2) 이미지 생성 (Image Generation)

이미지 생성 AI는 텍스트 설명이나 다른 이미지를 입력받아 새로운 이미지를 만들어냅니다. 핵심 기술로는 GAN(Generative Adversarial Network), VAE(Variational Autoencoder), 그리고 최근 주류를 이루는 Diffusion 모델이 있습니다.

Diffusion 모델은 이미지에 점진적으로 노이즈를 추가하는 과정(Forward Process)을 학습한 뒤, 역으로 노이즈에서 이미지를 복원하는 방식(Reverse Process)으로 이미지를 생성합니다. 이 방식이 GAN 대비 학습 안정성이 높고 다양한 이미지를 생성할 수 있어 현재 표준으로 자리 잡았습니다.

💡 Diffusion 모델의 원리 (간략)

1. Forward Process: 원본 이미지에 단계별로 가우시안 노이즈를 추가하여 완전한 노이즈로 변환
2. 노이즈 예측 학습: U-Net 구조의 신경망이 각 단계에서 추가된 노이즈를 예측하도록 학습
3. Reverse Process: 학습된 노이즈 예측을 이용해 순수 노이즈에서 이미지를 단계별로 복원
4. Guidance: CLIP 텍스트 인코더와 결합하여 텍스트 프롬프트로 이미지 생성 방향을 제어

(3) 오디오 생성 (Audio Generation)

오디오 생성 AI는 텍스트를 음성으로 변환(TTS, Text-To-Speech)하거나, 음악을 생성하거나, 사람의 목소리를 복제하는 등 다양한 작업을 수행합니다.

- TTS (Text-To-Speech): Microsoft의 SpeechT5, Coqui TTS, ElevenLabs
- 음성 인식 (ASR, Automatic Speech Recognition): OpenAI Whisper
- 음악 생성: Meta MusicGen, Google MusicLM
- 음성 변환 (Voice Conversion): 화자의 특성을 유지하면서 다른 화자 스타일로 변환

(4) 멀티모달 AI (Multimodal AI)

멀티모달 AI는 텍스트, 이미지, 오디오 등 여러 종류의 데이터를 동시에 처리하고 생성할 수 있는 모델입니다. GPT-4V, Google Gemini Ultra, Claude 3 Opus 등이 대표적인 멀티모달 AI입니다.

멀티모달 AI는 이미지를 보고 질문에 답하거나, 텍스트 설명을 읽고 이미지를 생성하거나, 동영상 분석하는 등 복합적인 작업을 수행할 수 있어 실용적인 활용 가능성이 매우 높습니다.

1.1.3 ChatGPT, DALL·E, Stable Diffusion 등 주요 모델 소개

생성형 AI 생태계를 이해하기 위해서는 이 분야를 선도하는 대표 모델들을 파악하는 것이 중요합니다. 다음 표는 현재 가장 널리 활용되고 있는 생성형 AI 모델들을 정리한 것입니다.

모델명	개발사/출처	주요 특징 및 활용
GPT-4 / GPT-4o	OpenAI	가장 강력한 LLM 중 하나. ChatGPT의 핵심 엔진 API 제공.
LLaMA 3	Meta AI	오픈소스 LLM. Hugging Face에서 제공. 로컬 실행 가능.
Mistral / Mixtral	Mistral AI	유럽 스타트업의 경량화 오픈소스 LLM. 높은 성능 대비 크기.
DALL·E 3	OpenAI	텍스트 프롬프트로 고품질 이미지 생성. ChatGPT와 통합.
Stable Diffusion	Stability AI	오픈소스 이미지 생성 모델. 로컬 실행·파인튜닝 가능.
SDXL / SDXL-Turbo	Stability AI	고해상도 이미지 생성. 실시간 생성 가능 버전 포함.

모델명	개발사/출처	주요 특징 및 활용
Whisper	OpenAI	다국어 음성 인식(STT). Hugging Face에서 오픈 제공.
SpeechT5	Microsoft / HF	텍스트-투-스피치(TTS). Hugging Face 공식 지
CLIP	OpenAI	이미지-텍스트 연결 모델. 이미지 검색/분류에 활
BLIP-2	Salesforce	이미지 캡셔닝 및 VQA(Visual Question Answer

1.1.4 생성형 AI가 웹과 연결될 때 어떤 서비스가 가능한가?

생성형 AI 기술은 그 자체로도 강력하지만, 웹 서비스와 결합될 때 비로소 실용적인 가치를 창출합니다. FastAPI와 같은 현대적인 웹 프레임워크를 통해 AI 모델을 API로 노출하면, 프론트엔드 웹 앱, 모바일 앱, 다른 백엔드 서비스 등 어디서든 AI 기능을 손쉽게 활용할 수 있습니다.

다음은 생성형 AI와 웹을 결합하여 구현할 수 있는 대표적인 서비스 사례들입니다.

서비스 유형	구체적인 활용 예시
AI 챗봇 서비스	LLM API를 호출하여 실시간 대화, 고객 지원, 업무 자동화
이미지 생성 플랫폼	사용자 텍스트 입력 → Stable Diffusion → 이미지 반환
문서 요약 서비스	PDF 업로드 → 텍스트 추출 → LLM 요약 → 결과 표시
다국어 번역 API	Helsinki-NLP 번역 모델 래핑 → REST API 제공
음성 분석 서비스	오디오 파일 업로드 → Whisper → 자막 텍스트 반환
감성 분석 API	리뷰 텍스트 입력 → 감성 분류 모델 → 긍정/부정 결과 반환
코드 리뷰 도우미	코드 입력 → LLM 분석 → 개선 사항·버그 제안
RAG 기반 Q&A	문서 인덱싱 + LLM 으로 문서 기반 질의응답 시스템

🔑 핵심 포인트

AI 모델 자체는 Python 함수(또는 클래스)이지만, 이를 웹 서비스로 노출하려면 HTTP 서버가 필요합니다.

FastAPI는 비동기(Async) 처리와 자동 API 문서 생성 기능을 갖춘 Python 웹 프레임워크로, AI 모델을 감싸는 마이크로서비스를 빠르게 구축하는 데 최적화되어 있습니다.

이 교재에서는 Hugging Face의 다양한 AI 모델을 FastAPI로 래핑하여 실제 서비스로 구현합니다.

1.2 Hugging Face란? (모델, 데이터셋, Spaces 등 개요)

Hugging Face는 단순한 AI 라이브러리 제공 업체를 넘어, 현재 AI 생태계에서 가장 중요한 플랫폼 중 하나로 성장했습니다. 'AI의 GitHub'라고도 불리는 Hugging Face는 연구자, 기업, 개발자 모두가 AI 모델과 데이터셋을 공유하고 협업하는 중심지입니다.

1.2.1 Hugging Face 플랫폼의 탄생 배경과 철학

Hugging Face는 2016년 Clément Delangue, Julien Chaumond, Thomas Wolf에 의해 챗봇 스타트업으로 창업되었습니다. 초기에는 청소년을 위한 대화형 AI 앱을 개발했지만, 2018년 BERT 모델을 쉽게 사용할 수 있는 라이브러리인 Transformers를 오픈소스로 공개하면서 AI 개발 도구 기업으로 방향을 전환했습니다.

Hugging Face의 핵심 철학은 '민주화(Democratization)'입니다. 최신 AI 기술을 특정 대형 기업만이 독점하는 것이 아니라, 누구나 접근하고 활용할 수 있어야 한다는 가치관 아래 대부분의 모델과 도구를 오픈소스로 제공합니다.

Hugging Face 연혁

- 2016년 - 챗봇 스타트업으로 창업 (뉴욕, 파리)
 - 2018년 - 🤖 Transformers 라이브러리 오픈소스 공개
 - 2019년 - Model Hub 런칭, 커뮤니티 모델 공유 시작
 - 2021년 - 4억 달러 시리즈 C 투자 유치 (기업가치 20억 달러)
 - 2022년 - Stable Diffusion 모델 호스팅, Spaces 확장
 - 2023년 - 등록 모델 30만 개 돌파, 데이터셋 8만 개 이상
 - 2024년 - 엔터프라이즈 서비스 확장, 멀티모달 모델 허브 강화
-

1.2.2 Model Hub: 30만 개 이상의 모델이 등록된 곳

Hugging Face Model Hub(<https://huggingface.co/models>)는 전 세계 연구자와 개발자들이 학습시킨 AI 모델을 공유하는 공간입니다. 2024년 기준으로 30만 개 이상의 모델이 등록되어 있으며, 매일 수백 개의 새 모델이 업로드됩니다.

Model Hub에서 모델을 검색할 때는 다음과 같은 필터를 활용할 수 있습니다.

- 태스크(Task): text-generation, text-classification, translation, summarization, image-classification, text-to-image, automatic-speech-recognition 등
- 라이브러리(Library): Transformers, Diffusers, Sentence Transformers 등
- 언어(Language): 한국어(Korean), 영어(English), 다국어(Multilingual) 등
- 라이선스(License): Apache 2.0, MIT, CC BY-SA 4.0 등
- 모델 크기: 파라미터 수 기준 필터링 가능

1.2.3 Dataset Hub: 공개 데이터셋 활용 방법

Hugging Face Dataset Hub(<https://huggingface.co/datasets>)는 8만 개 이상의 공개 데이터셋을 제공합니다. NLP, 컴퓨터 비전, 오디오 등 다양한 분야의 데이터셋을 datasets 라이브러리를 한 줄로 바로 사용할 수 있습니다.

```
# datasets 라이브러리로 데이터셋 불러오기
```

```
from datasets import load_dataset
```

```
# IMDB 영화 리뷰 데이터셋 로드
```

```
dataset = load_dataset('imdb')
```

```
print(dataset['train'][0])
```

```
# {'text': 'I love this movie...', 'label': 1}
```

```
# 한국어 데이터셋 예시
```

```
kor_dataset = load_dataset('klue', 'ynat')
```

1.2.4 Spaces: 모델을 배포할 수 있는 인터랙티브 웹 앱 호스팅

Hugging Face Spaces(<https://huggingface.co/spaces>)는 AI 모델 기반의 웹 데모 앱을 무료로 호스팅할 수 있는 서비스입니다. Gradio 또는 Streamlit 프레임워크로 만든 앱을 배포하면, 누구나 브라우저에서 직접 모델을 체험할 수 있습니다.

Spaces 특징	세부 내용
-----------	-------

Spaces 특징	세부 내용
지원 SDK	Gradio, Streamlit, Docker, 정적 HTML
무료 티어	CPU 기본 인스턴스 제공 (2 vCPU, 16GB RAM)
유료 티어	GPU 인스턴스 (T4, A10G, A100) 시간당 과금
지속 스토리지	데이터셋·모델 파일 영구 저장 가능
CI/CD	Git push 시 자동 빌드·배포
공개/비공개	Public 또는 Private 설정 가능

1.2.5 Hugging Face의 파이프라인과 🤖 Transformers 라이브러리

🤖 Transformers는 Hugging Face의 핵심 Python 라이브러리로, 수백 가지 사전 학습 모델을 단일 API로 사용할 수 있게 해줍니다. 특히 `pipeline()` 함수는 모델 로드, 전처리, 추론, 후처리를 한 번에 처리하는 고수준 인터페이스를 제공합니다.

```
from transformers import pipeline

# 1. 감성 분석 파이프라인
classifier = pipeline('sentiment-analysis')
result = classifier('I love using Hugging Face!')
# [{'label': 'POSITIVE', 'score': 0.9998}]

# 2. 텍스트 생성 파이프라인
generator = pipeline('text-generation', model='gpt2')
result = generator('Hello, I am a',
                    max_length=30, num_return_sequences=2)

# 3. 번역 파이프라인 (영어 → 한국어)
translator = pipeline('translation_en_to_ko',
                       model='Helsinki-NLP/opus-mt-en-ko')
result = translator('This is a FastAPI tutorial.')
```

4. 이미지 분류 파이프라인

```
img_classifier = pipeline('image-classification',  
                           model='google/vit-base-patch16-224')  
result = img_classifier('cat.jpg')
```

★ pipeline()이 내부적으로 하는 일

1. AutoTokenizer 또는 AutoFeatureExtractor로 입력 데이터 전처리
2. AutoModel로 해당 태스크에 맞는 사전 학습 모델 자동 로드
3. 모델 추론(Forward Pass) 실행
4. 결과 후처리 (확률값 계산, 디코딩 등)
5. 사용자 친화적인 딕셔너리 형태로 결과 반환

1.3 Hugging Face Hub 살펴보기

이 절에서는 Hugging Face Hub를 실제로 탐색하고 활용하는 방법을 상세히 설명합니다. 웹 인터페이스를 통한 모델 검색부터, Python 코드로 모델을 불러와 로컬에서 실행하는 전 과정을 다룹니다.

1.3.1 Model Card 이해하기

Hugging Face Hub에서 모델을 클릭하면 'Model Card'라고 불리는 상세 페이지가 나타납니다. Model Card는 모델의 설명서로, 다음과 같은 중요한 정보들을 담고 있습니다.

Model Card 구성 요소	내용 및 확인 사항
모델 설명 (Model Description)	모델의 목적, 아키텍처, 학습 데이터 요약
사용 방법 (How to use)	transformers를 이용한 코드 예제
태스크 및 성능 (Evaluation Results)	벤치마크 점수, 평가 데이터셋 정보
한계 및 주의사항 (Limitations)	모델의 편향, 한계, 부적절한 사용 사례
라이선스 (License)	상업적 사용 가능 여부, 재배포 조건
토큰 인증 필요 여부	gated 모델의 경우 HF 토큰 필요
모델 파일 목록 (Files)	config.json, tokenizer, pytorch_model.bin 등

⚠ Gated 모델 주의사항

일부 모델(예: Meta LLaMA, Mistral)은 'Gated' 설정이 되어 있어 다운로드하려면 Hugging Face 계정으로 로그인하고 Access Request를 신청해야 합니다. 승인 후 `huggingface-cli login` 또는 `HF_TOKEN` 환경변수로 인증이 필요합니다.

```
$ huggingface-cli login
```

```
# 또는
```

```
$ export HF_TOKEN='hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX'
```

1.3.2 원하는 모델 검색 및 다운로드 방법

Hugging Face Hub에서 모델을 검색하고 다운로드하는 방법은 크게 세 가지입니다.

방법 1: 웹 인터페이스 검색

<https://huggingface.co/models>에 접속하여 검색창에 원하는 모델명이나 태스크를 입력합니다. 왼쪽 필터 패널에서 태스크, 라이브러리, 언어, 라이선스 등을 설정하여 범위를 좁힐 수 있습니다.

방법 2: `huggingface_hub` 라이브러리로 검색

```
from huggingface_hub import HfApi

api = HfApi()

# 태스크로 모델 검색
models = api.list_models(
    filter='text-generation',
    sort='downloads',
    direction=-1,    # 내림차순
    limit=10
)

for model in models:
    print(f'{model.id:50s} downloads: {model.downloads}')
```

방법 3: `snapshot_download` 또는 `hf_hub_download`

```
from huggingface_hub import snapshot_download, hf_hub_download

# 모델 전체 다운로드 (로컬 캐시에 저장)
local_dir = snapshot_download(
    repo_id='distilbert-base-uncased-finetuned-sst-2-english'
)
```

```
print(f'Downloaded to: {local_dir}')
```

```
# 특정 파일만 다운로드
```

```
file_path = hf_hub_download(
    repo_id='bert-base-uncased',
    filename='config.json'
)
```

1.3.3 pipeline, transformers, datasets 기본 구조 실습

이 절에서는 Hugging Face의 세 가지 핵심 라이브러리인 transformers, datasets, pipeline을 실제 코드로 실습합니다. 이 세 라이브러리는 이 교재 전체에서 반복적으로 사용되므로 기본 구조를 확실히 이해하는 것이 중요합니다.

실습 1: transformers 기본 사용 흐름

```
# 설치: pip install transformers torch
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
```

```
import torch
```

```
# 1. 토크나이저 로드
```

```
model_name = 'distilbert-base-uncased-finetuned-sst-2-english'
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
# 2. 모델 로드
```

```
model = AutoModelForSequenceClassification.from_pretrained(model_name)
```

```
# 3. 입력 전처리 (토크나이징)
```

```
inputs = tokenizer('I really love this product!',
```

```
    return_tensors='pt')
```

```
# inputs: {'input_ids': tensor([[...]]), 'attention_mask': tensor([[...]])}
```

```
# 4. 모델 추론
```

```
with torch.no_grad():
```

```

outputs = model(**inputs)

# 5. 결과 후처리
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)
predicted_class = probabilities.argmax().item()
labels = model.config.id2label
print(f'Predicted: {labels[predicted_class]}')
print(f'Confidence: {probabilities.max().item():.4f}')

```

실습 2: datasets 라이브러리 기본 사용

```

# 설치: pip install datasets
from datasets import load_dataset, DatasetDict

# 1. 데이터셋 로드
dataset = load_dataset('imdb')
print(type(dataset))      # DatasetDict
print(dataset)            # 분할 정보 출력

# 2. 데이터 확인
print(dataset['train'][0]) # 첫 번째 훈련 데이터
print(dataset['train'].features) # 컬럼 정보

# 3. 데이터 필터링
positive_reviews = dataset['train'].filter(
    lambda x: x['label'] == 1
)
print(f'긍정 리뷰 개수: {len(positive_reviews)}')

# 4. 로컬 파일 로드
local_dataset = load_dataset('csv', data_files='my_data.csv')

```

1.3.4 로컬에 모델 캐싱하고 사용하는 흐름 소개

Hugging Face 모델은 처음 `from_pretrained()`를 호출할 때 자동으로 인터넷에서 다운로드되어 로컬 캐시 디렉터리에 저장됩니다. 이후 동일한 모델을 불러올 때는 캐시에서 즉시 로드되므로, 인터넷 연결이 불안정한 환경이나 반복 실행 시 매우 유용합니다.

■ 기본 캐시 경로

Linux/macOS: `~/.cache/huggingface/hub/`

Windows: `C:\Users\ <사용자명>\.cache\huggingface\hub\`

환경변수로 캐시 경로 변경:

```
$ export HF_HOME=/data/my_hf_cache
```

```
$ export TRANSFORMERS_CACHE=/data/models
```

```
import os

from transformers import AutoTokenizer, AutoModelForCausalLM

# 캐시 경로 커스터마이징
os.environ['HF_HOME'] = '/data/huggingface_cache'

# 오프라인 모드 (캐시된 모델만 사용)
os.environ['TRANSFORMERS_OFFLINE'] = '1'

# local_files_only=True로 로컬 캐시에서만 로드
model = AutoModelForCausalLM.from_pretrained(
    'gpt2',
    local_files_only=True,
    cache_dir='/data/huggingface_cache'
)

# 특정 디렉터리에 모델 저장 후 재사용
model.save_pretrained('./my_local_model')
tokenizer.save_pretrained('./my_local_model')
```

```
# 로컬 디렉터리에서 불러오기
```

```
loaded_model = AutoModelForCausalLM.from_pretrained('./my_local_model')
```

1.4 Python 환경 세팅과 FastAPI 소개

본격적인 AI 웹 개발을 시작하기 전에, 안정적이고 재현 가능한 개발 환경을 구축하는 것이 매우 중요합니다. 이 절에서는 Python 버전 선택부터 가상 환경 생성, 필수 패키지 설치, 그리고 FastAPI의 핵심 특징까지 단계적으로 설명합니다.

1.4.1 Python 3.10 이상 버전 권장

이 교재는 Python 3.10 이상을 권장합니다. Python 3.10부터 도입된 구조적 패턴 매칭 (match-case), 개선된 에러 메시지, 타입 힌트 문법 개선 등이 현대적인 Python 코드 작성에 도움이 됩니다. 또한 FastAPI, PyTorch, Transformers 등 주요 AI 라이브러리들이 3.10 이상에서 더 안정적으로 동작합니다.

```
# 현재 Python 버전 확인
```

```
$ python --version
```

```
Python 3.11.8
```

```
# 또는
```

```
$ python3 --version
```

```
Python 3.11.8
```

💡 Python 버전 관리 도구 추천

pyenv: 여러 Python 버전을 손쉽게 설치·전환할 수 있는 CLI 도구

```
$ curl https://pyenv.run | bash
```

```
$ pyenv install 3.11.8
```

```
$ pyenv global 3.11.8
```

Windows에서는 공식 사이트(<https://python.org>)에서 인스톨러를 다운로드하거나 Microsoft Store에서 설치하는 것을 권장합니다.

1.4.2 가상 환경 만들기 (venv, conda)

가상 환경(Virtual Environment)은 프로젝트별로 독립적인 Python 패키지 환경을 제공합니다. 여러 프로젝트가 서로 다른 버전의 패키지를 사용할 때 충돌을 방지하고, 환경을 팀원과 공유하거나 배포 시 재현성을 보장합니다.

방법 1: venv (Python 내장, 권장)

```
# 프로젝트 디렉터리 생성 및 이동
$ mkdir fastapi-ai-project
$ cd fastapi-ai-project

# 가상 환경 생성
$ python -m venv venv

# 가상 환경 활성화
# macOS / Linux
$ source venv/bin/activate

# Windows (Command Prompt)
$ venv\Scripts\activate.bat

# Windows (PowerShell)
$ venv\Scripts\Activate.ps1

# 활성화 확인 (프롬프트 앞에 (venv) 표시)
(venv) $ python --version
Python 3.11.8

# 가상 환경 비활성화
(venv) $ deactivate
```

방법 2: conda (Anaconda / Miniconda)

```
# conda 환경 생성
$ conda create -n fastapi-ai python=3.11

# 환경 활성화
$ conda activate fastapi-ai

# 환경 목록 확인
$ conda env list

# 환경 비활성화
$ conda deactivate

# 환경 삭제
$ conda env remove -n fastapi-ai
```

1.4.3 필수 패키지 설치

이 교재에서 사용하는 핵심 패키지들을 설치합니다. 프로젝트 루트 디렉터리에 requirements.txt 파일을 만들어 패키지 목록을 관리하는 것이 좋습니다.

requirements.txt 작성

```
# requirements.txt

# 웹 프레임워크
fastapi==0.111.0
uvicorn[standard]==0.29.0
python-multipart==0.0.9

# AI / ML 라이브러리
torch==2.3.0
transformers==4.41.0
datasets==2.19.0
huggingface_hub==0.23.0
```

```
accelerate==0.30.0
```

```
diffusers==0.27.2
```

```
# 유틸리티
```

```
Pillow==10.3.0
```

```
numpy==1.26.4
```

```
pydantic==2.7.1
```

```
python-dotenv==1.0.1
```

```
httpx==0.27.0
```

```
aiofiles==23.2.1
```

```
# 패키지 일괄 설치
```

```
(venv) $ pip install -r requirements.txt
```

```
# 또는 개별 설치
```

```
(venv) $ pip install fastapi uvicorn transformers torch
```

```
# 설치된 패키지 확인
```

```
(venv) $ pip list
```

```
# 현재 환경의 패키지 목록 저장
```

```
(venv) $ pip freeze > requirements.txt
```

💡 GPU 가속을 위한 PyTorch 설치

GPU(CUDA)를 사용하려면 CUDA 버전에 맞는 PyTorch를 설치해야 합니다.

PyTorch 공식 사이트(<https://pytorch.org/get-started/locally/>)에서 자신의 환경에 맞는 설치 명령어를 확인하세요.

CUDA 12.1 기준:

```
$ pip install torch torchvision torchaudio --index-url \
    https://download.pytorch.org/whl/cu121
```

CPU 전용 (가벼운 실습용):

```
$ pip install torch torchvision torchaudio --index-url \
https://download.pytorch.org/whl/cpu
```

1.4.4 FastAPI란? (비동기 처리, 경량 API 서버, Swagger UI 지원)

FastAPI는 Sebastian Ramirez가 개발한 Python 기반의 현대적인 웹 API 프레임워크입니다. 2018년 처음 공개된 이후 빠른 성능, 간결한 코드, 자동 문서 생성 등의 강점으로 빠르게 성장하여 현재 Python 웹 프레임워크 중 Django, Flask와 함께 3대 프레임워크로 자리 잡았습니다.

FastAPI의 핵심 특징은 다음과 같습니다.

특징	상세 설명
고성능 (High Performance)	Starlette(비동기 웹 서버)와 Pydantic(데이터 검증) 기반. Node.js, C와 유사한 수준의 처리 속도.
비동기 지원 (Async/Await)	Python async/await 문법으로 I/O 바운드 작업을 비블로킹으로 처리. AI 모델 추론 중 다른 요청 처리 가능.
자동 문서 생성	코드 작성만으로 Swagger UI와 ReDoc 문서가 자동 생성됨. 별도 문서 작성 불필요.
타입 힌트 기반	Python 타입 힌트로 요청/응답 데이터를 선언하면 자동 검증·직렬화·역직렬화 처리.
Pydantic 통합	강력한 데이터 모델 정의 및 유효성 검사. JSON 직렬화 자동 처리.
의존성 주입 (DI)	내장 DI 시스템으로 DB 세션, 인증, AI 모델 등을 깔끔하게 관리.
표준 준수	OpenAPI(Swagger) 및 JSON Schema 표준 완전 지원.

FastAPI와 다른 Python 웹 프레임워크를 비교하면 다음과 같습니다.

항목	FastAPI	Flask
출시 연도	2018년	2010년
비동기 지원	네이티브 지원	△ 플러그인 필요
자동 문서화	Swagger + ReDoc	직접 구현 필요

항목	FastAPI	Flask
타입 검증	Pydantic 내장	△ marshmallow 등
성능 (req/sec)	매우 빠름 (Starlette)	중간 수준
학습 곡선	중간	낮음
AI 프로젝트 적합도	매우 적합	보통

1.4.5 Swagger, ReDoc 문서 자동 생성 기능 미리 보기

FastAPI의 가장 편리한 기능 중 하나는 코드만 작성하면 API 문서가 자동으로 생성된다는 점입니다. uvicorn 서버를 실행한 후 브라우저에서 아래 경로에 접속하면 됩니다.

문서 URL	설명
http://127.0.0.1:8000/docs	Swagger UI - 대화형 API 문서, 직접 요청 실행 가능
http://127.0.0.1:8000/redoc	ReDoc - 더 깔끔한 읽기용 API 문서
http://127.0.0.1:8000/openapi.json	OpenAPI 스펙 JSON 파일 (자동 생성)

1.5 Hello World: FastAPI 웹 서버 띄우기

이제 이론에서 실습으로 넘어갑니다. 이 절에서는 FastAPI로 첫 번째 웹 서버를 만들고, Swagger UI에서 API를 테스트하며, 브라우저와 curl 명령어로 결과를 확인하는 전 과정을 단계별로 진행합니다.

1.5.1 FastAPI 앱 구조 만들기

FastAPI 프로젝트의 기본 디렉터리 구조를 먼저 만들겠습니다. 처음에는 단순하게 시작하고, 이후 챕터에서 점진적으로 구조를 확장해 나갈 것입니다.

fastapi-ai-project/	
— main.py	# FastAPI 앱 진입점
— requirements.txt	# 의존 패키지 목록
— .env	# 환경변수 (HF_TOKEN 등)
— routers/	# 라우터 모듈 디렉터리
— __init__.py	
— models/	# Pydantic 데이터 모델
— __init__.py	
— services/	# AI 모델 서비스 로직
— __init__.py	

1.5.2 첫 번째 라우터 작성: /hello

main.py 파일을 생성하고 첫 번째 FastAPI 애플리케이션을 작성합니다. 아래 코드를 한 줄씩 이해하면서 입력해 보세요.

main.py
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional
FastAPI 앱 인스턴스 생성

```
app = FastAPI(
    title='HuggingFace AI API',
    description='FastAPI와 Hugging Face로 만드는 AI 웹 서비스',
    version='1.0.0',
)

# -----
# 1. 가장 기본: GET /hello
# -----
@app.get('/hello')
def hello_world():
    return {'message': 'Hello, FastAPI World!'}

# -----
# 2. 경로 매개변수 사용: GET /hello/{name}
# -----
@app.get('/hello/{name}')
def hello_name(name: str):
    return {'message': f'Hello, {name}! FastAPI에 오신 걸 환영합니다.'}

# -----
# 3. 쿼리 매개변수: GET /greet?name=홍길동&lang=ko
# -----
@app.get('/greet')
def greet(
    name: str = 'World',
    lang: Optional[str] = 'en'
):
    greetings = {'ko': '안녕하세요', 'en': 'Hello', 'ja': 'こんにちは'}
    greeting = greetings.get(lang, 'Hello')
    return {'message': f'{greeting}, {name}!', 'language': lang}

# -----
# 4. POST 요청과 Pydantic 모델
# -----
```

```

class EchoRequest(BaseModel):
    text: str
    repeat: int = 1

class EchoResponse(BaseModel):
    original: str
    echoed: str
    count: int

@app.post('/echo', response_model=EchoResponse)
def echo_text(request: EchoRequest):
    return EchoResponse(
        original=request.text,
        echoed=' '.join([request.text] * request.repeat),
        count=request.repeat
    )

```

1.5.3 Swagger UI로 결과 확인하기

코드 각 부분의 의미를 상세히 살펴보겠습니다.

코드 요소	역할 및 설명
FastAPI(title= ..., ...)	앱 메타데이터 설정. Swagger UI 상단에 표시됨.
@app.get('/hello')	HTTP GET 메서드로 /hello 경로에 라우터 등록.
@app.post('/echo')	HTTP POST 메서드 라우터 등록.
def hello_world():	동기 함수. 단순 작업에 사용. async def도 사용 가능.
name: str	경로/쿼리 매개변수 타입 선언. 자동 검증 적용.
BaseModel	Pydantic 모델. 요청/응답 데이터 구조 정의.
response_model=EchoResponse	응답 데이터 타입 지정. 자동 문서화에 활용.
Optional[str]	None 허용 타입. 선택적 쿼리 매개변수에 사용.

1.5.4 서버 실행

FastAPI 앱은 ASGI(Asynchronous Server Gateway Interface) 서버를 통해 실행됩니다. 이 교재에서는 uvicorn을 사용합니다.

기본 실행 (개발 환경)

```
(venv) $ uvicorn main:app --reload
```

옵션 설명:

main:app - 'main.py' 파일의 'app' 인스턴스를 실행

--reload - 코드 변경 시 자동 재시작 (개발 시에만 사용)

호스트와 포트 지정

```
(venv) $ uvicorn main:app --host 0.0.0.0 --port 8080 --reload
```

프로덕션 환경 (workers 추가)

```
(venv) $ uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

정상 실행 시 출력:

```
INFO: Will watch for changes in these directories: ['/path/to/project']
```

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

```
INFO: Started reloader process [12345] using WatchFiles
```

```
INFO: Started server process [12346]
```

```
INFO: Waiting for application startup.
```

```
INFO: Application startup complete.
```

💡 Python 코드에서 서버 실행하기

스크립트로 실행하고 싶다면 main.py 하단에 다음 코드를 추가하세요.

```
if __name__ == '__main__':
    import uvicorn
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

이후 python main.py 명령으로 실행할 수 있습니다.

1.5.5 실제 브라우저에서 API 호출 및 결과 확인

서버가 실행 중인 상태에서 다양한 방법으로 API를 테스트해 보겠습니다.

방법 1: 브라우저에서 직접 접속 (GET 요청)

브라우저 주소창에 입력
http://127.0.0.1:8000/hello
결과: {"message": "Hello, FastAPI World!"}
http://127.0.0.1:8000/hello/홍길동
결과: {"message": "Hello, 홍길동! FastAPI에 오신 걸 환영합니다."}
http://127.0.0.1:8000/greet?name=김철수&lang=ko
결과: {"message": "안녕하세요, 김철수!", "language": "ko"}

방법 2: curl 명령어로 테스트

GET 요청
\$ curl http://127.0.0.1:8000/hello
{"message": "Hello, FastAPI World!"}
POST 요청 (JSON 데이터 전송)
\$ curl -X POST http://127.0.0.1:8000/echo ₩
-H 'Content-Type: application/json' ₩
-d '{"text": "FastAPI", "repeat": 3}'
결과:
{
"original": "FastAPI",
"echoed": "FastAPI FastAPI FastAPI",
"count": 3
}

방법 3: Swagger UI에서 대화형 테스트

브라우저에서 <http://127.0.0.1:8000/docs>에 접속하면 Swagger UI가 열립니다. 각 API 엔드포인트를 클릭하면 요청 파라미터와 응답 스키마가 자동으로 표시되며, 'Try it out' 버튼을 클릭하여 직접 API를 실행해볼 수 있습니다.

- Swagger UI 사용법: 엔드포인트 클릭 → 'Try it out' 버튼 → 파라미터 입력 → 'Execute' 클릭 → 응답 확인
- ReDoc(<http://127.0.0.1:8000/redoc>)은 더 깔끔한 문서 뷰를 제공하지만 직접 실행 기능은 없습니다.

방법 4: Python requests 라이브러리로 테스트

```
# 설치: pip install requests

import requests

base_url = 'http://127.0.0.1:8000'

# GET /hello
response = requests.get(f'{base_url}/hello')
print(response.json())
# {'message': 'Hello, FastAPI World!'}

# GET /greet?name=홍길동&lang=ko
response = requests.get(
    f'{base_url}/greet',
    params={'name': '홍길동', 'lang': 'ko'}
)
print(response.json())
# {'message': '안녕하세요, 홍길동!', 'language': 'ko'}

# POST /echo
response = requests.post(
    f'{base_url}/echo',
    json={'text': 'FastAPI', 'repeat': 3}
```

```
)  
print(response.json())  
# {'original': 'FastAPI', 'echoed': 'FastAPI FastAPI FastAPI', 'count': 3}
```

1.6 종합 실습: 간단한 AI Hello World API

지금까지 배운 내용을 토대로, Hugging Face의 감성 분석 모델을 FastAPI로 래핑한 간단한 AI API를 만들어보겠습니다. 이것이 이 교재 전체 흐름의 축소판입니다.

```
# ai_hello.py  
from fastapi import FastAPI, HTTPException  
from pydantic import BaseModel  
from transformers import pipeline  
import uvicorn  
  
app = FastAPI(  
    title='AI Hello World API',  
    description='Hugging Face 감성 분석 모델을 FastAPI로 래핑한 첫 AI API',  
    version='1.0.0',  
)  
  
# 앱 시작 시 모델 한 번만 로드 (전역 변수)  
sentiment_analyzer = None  
  
@app.on_event('startup')  
async def load_model():  
    global sentiment_analyzer  
    print('감성 분석 모델 로딩 중...')  
    sentiment_analyzer = pipeline(  
        'sentiment-analysis',  
        model='distilbert-base-uncased-finetuned-sst-2-english'  
    )  
    print('모델 로딩 완료!')
```

```
# 요청 모델
```

```
class SentimentRequest(BaseModel):
```

```
    text: str
```

```
class Config:
```

```
    json_schema_extra = {
```

```
        'example': {'text': 'I love using FastAPI and Hugging Face!'}
```

```
    }
```

```
# 응답 모델
```

```
class SentimentResponse(BaseModel):
```

```
    text: str
```

```
    label: str    # POSITIVE 또는 NEGATIVE
```

```
    score: float  # 신뢰도 (0.0 ~ 1.0)
```

```
    interpretation: str
```

```
@app.post('/analyze', response_model=SentimentResponse)
```

```
async def analyze_sentiment(request: SentimentRequest):
```

```
    if sentiment_analyzer is None:
```

```
        raise HTTPException(status_code=503,
```

```
                             detail='모델이 아직 로딩 중입니다.')
```

```
    result = sentiment_analyzer(request.text)[0]
```

```
    interpretation = ('긍정적인 텍스트입니다.'
```

```
                     if result['label'] == 'POSITIVE'
```

```
                     else '부정적인 텍스트입니다.')
```

```
    return SentimentResponse(
```

```
        text=request.text,
```

```
        label=result['label'],
```

```
        score=round(result['score'], 4),
```

```
        interpretation=interpretation
```

```
)
```



```
@app.get('/health')
async def health_check():
    return {
        'status': 'healthy',
        'model_loaded': sentiment_analyzer is not None
    }

if __name__ == '__main__':
    uvicorn.run('ai_hello:app', host='0.0.0.0', port=8000, reload=True)
```

서버 실행

```
(venv) $ uvicorn ai_hello:app --reload
```

다른 터미널에서 테스트

```
$ curl -X POST http://127.0.0.1:8000/analyze \
-H 'Content-Type: application/json' \
-d '{"text": "I love this FastAPI tutorial!"}'
```

응답:

```
{
  "text": "I love this FastAPI tutorial!",
  "label": "POSITIVE",
  "score": 0.9998,
  "interpretation": "긍정적인 텍스트입니다."
}
```

🎯 1장 학습 정리

전통적 AI와 생성형 AI의 차이를 이해했습니다.

Hugging Face 플랫폼의 Model Hub, Dataset Hub, Spaces를 파악했습니다.

pipeline() 함수로 다양한 AI 태스크를 간단하게 실행할 수 있습니다.

Python 가상 환경을 설정하고 필수 패키지를 설치했습니다.

FastAPI의 핵심 특징(비동기, 자동 문서화, 타입 힌트)을 이해했습니다.

첫 번째 FastAPI 서버를 실행하고 Swagger UI에서 테스트했습니다.
Hugging Face 모델을 FastAPI로 래핑한 간단한 AI API를 완성했습니다.

연습 문제

다음 문제들을 풀어보면서 1장에서 배운 개념을 확인해 보세요.

- 다음 중 생성형 AI(Generative AI)의 특징으로 옳지 않은 것은?
 - 1.1. a. 새로운 데이터를 생성할 수 있다.
 - 1.2. b. 대규모 사전 학습 데이터가 필요하다.
 - 1.3. c. 입력 데이터를 정해진 클래스 중 하나로 분류하는 것만 가능하다.
 - 1.4. d. 텍스트, 이미지, 오디오 생성이 모두 포함된다.
- Hugging Face의 pipeline() 함수에 대한 설명으로 옳은 것을 모두 고르세요.
 - 2.1. a. 입력 전처리, 모델 추론, 후처리를 자동으로 수행한다.
 - 2.2. b. pipeline()을 사용하면 로컬에 모델을 저장할 수 없다.
 - 2.3. c. task 파라미터로 sentiment-analysis, text-generation 등을 지정한다.
 - 2.4. d. 반드시 GPU가 있어야 실행 가능하다.
- FastAPI에서 자동으로 생성되는 API 문서의 기본 URL을 두 가지 쓰세요.
- 다음 FastAPI 코드에서 오류를 찾고 수정하세요.

```
@app.get('/items/{item_id}')  
def get_item(item_id):  
    return {'item_id': item_id}
```

- Hugging Face에서 한국어 감성 분석 모델을 검색하고, 해당 모델을 pipeline()으로 불러와 '오늘 날씨가 정말 좋네요!'를 분석하는 코드를 작성하세요.

1장 요약

이 장에서는 AI 웹 개발 여정의 첫걸음으로, 생성형 AI의 개념과 Hugging Face 플랫폼, 그리고 FastAPI 기반의 개발 환경 구축을 학습했습니다.

- 생성형 AI는 텍스트, 이미지, 오디오 등 새로운 콘텐츠를 생성하는 AI 기술로, 트랜스포머 아키텍처와 대규모 사전 학습을 기반으로 합니다.
- Hugging Face는 30만 개 이상의 모델과 8만 개 이상의 데이터셋을 제공하는 AI 생태계의 핵심 플랫폼이며, Transformers, datasets, Spaces 등 강력한 도구를 무료로 제공합니다.
- pipeline() 함수는 AI 모델을 가장 간단하게 활용할 수 있는 고수준 인터페이스로, 전처리부터 추론, 후처리까지 자동으로 처리합니다.
- FastAPI는 비동기 처리, 자동 문서화, 타입 힌트 기반 검증을 지원하는 현대적인 Python 웹 프레임워크로, AI 모델 서빙에 최적화되어 있습니다.
- uvicorn을 통해 FastAPI 서버를 실행하고, Swagger UI(<http://localhost:8000/docs>)에서 API를 대화형으로 테스트할 수 있습니다.

□ 다음 장 예고: 제2장 텍스트 분류 & 감성 분석 API

2장에서는 Hugging Face의 다양한 텍스트 분류 모델을 활용하여 본격적인 AI API를 구현합니다.

주요 내용:

- 감성 분석(Sentiment Analysis) 모델 심층 이해
- 한국어 NLP 모델 활용 방법
- FastAPI 라우터 분리 및 앱 구조화
- 비동기 모델 추론으로 성능 최적화
- 예외 처리 및 입력 검증 강화
- Docker로 AI API 컨테이너화